CHRISCO Youth Aflame
CHRISCO DIGITAL ACADEMY | CHRISCO FELLOWSHIP OF CHURCHES

SWE & LLM Mastery Course Handbook

"Build real, AI-powered software with modern engineering."



Software Engineering & AI Systems Track | 10 Weeks | Intermediate Self-Paced Study Edition

Prepared by CHRISCO Youth Aflame | chriscoyouthaflame2025@gmail.com

Course Foreword



We are living through a rare shift: large language models (LLMs) have made it possible to build genuinely intelligent software faster than ever. Combine solid software engineering with the ability to harness LLMs, and you become the kind of builder companies are desperate to hire, or the founder who ships products alone.

This book assumes you can already code a little (or are learning). It focuses on how to think like an engineer and how to build with AI, not on one specific language. The principles here will outlast any single tool or model.

THE PRACTITIONER PROMISE

You will learn software engineering foundations, how large language models work, prompt and API skills, and how to design, build, test and deploy an AI-powered application of your own.

The goal is a working AI-powered app you build yourself. Nothing proves skill, to employers, clients, or yourself, like a real thing that works. Build as you read.

Published by CHRISCO Digital Academy under CHRISCO Youth Aflame (*CHRISCO Fellowship of Churches*), this 8-module workbook equips youth and emerging leaders with practical, ethical, income-ready skills. Complete every chapter drill in a dedicated workbook and reach our mentorship desk anytime at chriscoyouthaflame2025@gmail.com.



Key Outcomes

01

Software engineering basics: Engineering = reliable, maintainable, problem-solving code.

02

Prompt engineering: Prompts are part of your app's code.

03

AI-powered app development: Use the LLM as one component of solid engineering.

04

Real-world projects: Ship one complete, live AI-powered app.

Across 10 Weeks of structured practice, learners progress from core principles to real-world execution. Every module pairs concise frameworks with an immediate hands-on drill so knowledge turns into demonstrable competence.

Graduates finish with a tangible portfolio deliverable in SWE & LLM Mastery, ready to serve clients, employers, ministries, and communities with excellence and integrity.

Table of Contents

Course Foreword & Key Outcomes 02

01 Foundations & Core Mindset 05

02 Core Frameworks & Strategy 07

03 Execution & Applied Mastery 10

04 Scaling, Capstone & Excellence 12

05 8-Module Syllabus & Drill Matrix 14

06 90-Day Action Plan & Stewardship 15



01
Foundations & Core Mindset

Module 1: Software engineering basics | Module 2: Understanding LLMs



1.1 Software engineering basics

Software engineering is more than writing code, it's building software that is reliable, maintainable and solves real problems. That means breaking problems into clear parts, writing clean and understandable code, using version control (like Git) to track changes, and thinking about how pieces fit together. These habits separate a hobbyist from a professional engineer.

Key foundations include understanding data structures and logic, reading and reusing others' code, debugging systematically, and collaborating using tools like Git and GitHub. You don't need to know everything, you need strong fundamentals and the ability to learn continuously, because technology changes fast. Good engineering thinking is the durable skill; specific tools come and go.

Real-World Example: Two coders build the same feature. One writes messy code that only they understand; the other writes clean, organised code with clear names and version control. Six months later, only the second can easily fix and extend it, that's engineering, not just coding.

Hands-On Practice: Set up Git and a GitHub account. Put a small piece of code (even a simple script) into a repository, making at least two commits.

Core Takeaways: Engineering = reliable, maintainable, problem-solving code. • Use version control (Git) from the start. • Strong fundamentals plus continuous learning win.

1.2 Understanding LLMs

Large language models are AI systems trained on vast amounts of text to predict and generate human-like language. They can write, summarise, translate, answer questions, and even help write code. Crucially, understand what they are: powerful pattern-predictors, not databases of facts, which is why they can be fluent yet occasionally confidently wrong ('hallucinate').

As a builder, you don't need to train these models yourself, you use existing ones (like those from OpenAI, Anthropic, Google and open-source options) via APIs. Knowing their strengths (language tasks, reasoning help, code assistance) and limits (accuracy, up-to-date facts, cost per use) lets you design smart applications that use LLMs where they shine and guard against where they fail.

01

Practitioner Case

A developer building a customer-support tool used an LLM to draft helpful replies but added a step to pull real, verified answers from the company's own documents, combining the model's fluency with reliable facts to avoid confident wrong answers.

02

Action Drill

Use an LLM (via a chat interface) for three different tasks: writing, summarising, and helping with code. Note where it excelled and where it erred.

03

Key Standard

LLMs predict language, fluent but not always factual. You use existing models via APIs, not train them. Design around their strengths and guard their limits.



02
Core Frameworks & Strategy

Module 3: Prompt engineering | Module 4: Building with APIs

2.1 Prompt engineering

Prompt engineering is the skill of instructing an LLM to get reliable, high-quality output, essential when building AI apps. A strong prompt provides clear context, a specific task, the desired format, and often examples. In applications, prompts are part of your code: well-crafted, tested prompts are the difference between an AI feature that delights users and one that frustrates them.

Techniques include giving the model a role, breaking complex tasks into steps, providing examples of the desired output (few-shot prompting), and instructing it to reason step by step for harder problems. You also learn to handle and validate the model's responses in code. Treat prompts as you would any code: iterate, test with many inputs, and refine until they're robust.

Core Principle

Prompts are part of your app's code.

Field Case Study

In an app that categorises customer messages, the developer's first prompt gave inconsistent results. By adding a clear role, three examples, and a strict output format, the categorisation became reliable enough to ship, the prompt itself was the fix.

Applied Exercise

Write a prompt that reliably makes an LLM output a specific format (e.g. always return JSON with certain fields). Test it on several different inputs.

Mastery Check

Use role, context, examples and clear format. • Iterate and test prompts against many inputs.

2.2 Three-Step Execution Rhythm

Step 1	Prepare & Frame Prompts are part of your app's code.	Practice Focus Audit your current approach before acting.
Step 2	Execute Deliberately Use role, context, examples and clear format.	Practice Focus Focus on clarity, calm pace, and measurable value.
Step 3	Review & Refine Iterate and test prompts against many inputs.	Practice Focus Record outcomes and sharpen one habit per rep.

2.3 Building with APIs

APIs let your software talk to LLMs and other services programmatically. Instead of a person typing into a chat, your code sends a request (with your prompt and settings) to the model's API and receives a response to use in your app. Learning to authenticate, send requests, handle responses, and manage errors and rate limits is the core skill for building AI-powered software.

You'll also work with other APIs, databases, payment, external data, since real apps combine many services. Key professional concerns include keeping API keys secret (never in public code), handling failures gracefully, and being mindful of cost (LLM APIs charge per use). Mastering API integration is what lets you assemble powerful applications from existing building blocks.

01

Your code calls LLMs and services via APIs.

02

Handle auth, errors, rate limits and cost.

03

Never expose API keys in public code.

2.4 Real-World Case & Lab Assignment

Practitioner Case Study

A developer built a study-helper app where the user's question is sent via API to an LLM, and the answer is displayed in the app. The whole 'intelligence' came from an API call, he focused on the experience around it, not building a model.

Your Practical Assignment

Write code that sends a prompt to an LLM API and prints the response. Add basic error handling for when the request fails. Keep your API key secret.

CHRISCO Youth Aflame Study Principle: "Your code calls LLMs and services via APIs. Consistent execution in small daily assignments builds authority that cannot be faked."



03
Execution & Applied Mastery

Module 5: AI-powered app development | Module 6: Testing and deployment

3.1 AI-powered app development

Now you combine engineering and LLMs into a real application. This means designing the app: what problem it solves, the user experience, the flow of data, and where the LLM fits. Good AI apps use the model as one component within solid engineering, with a clear interface, reliable data handling, and thoughtful prompts, rather than just a thin wrapper around a chatbot.

Think about architecture: a front end users interact with, back-end logic, data storage, and the LLM integration, all working together. Consider the user's journey and edge cases (what if the AI returns something wrong or nothing?). The best AI products feel seamless because good engineering surrounds and supports the AI, handling the messy reality the model alone can't.



Case: A team built an app that turns a user's rough notes into organised meeting minutes: the front end captures notes, the back end sends a carefully-prompted request to an LLM, validates the result, and saves it. Solid engineering around the AI made it genuinely useful.

Action Drill: Design (on paper) an AI-powered app: the problem it solves, the user flow, where the LLM fits, and how you'll handle wrong or empty AI responses.

3.2 Testing and deployment

Software isn't done when it works on your machine, it must work reliably for users. Testing means checking that your app behaves correctly, including with unexpected inputs and when the AI misbehaves. Write tests for your logic, and manually test the whole user experience. With AI features, also test that your prompts and validation handle a wide range of inputs gracefully.

Deployment means putting your app where users can access it, on a hosting platform or cloud service. You'll learn to deploy your application, manage environment settings (like API keys securely), and monitor it once live. Shipping a real, deployed app, not just code on your laptop, is a major milestone and exactly what proves professional capability.



Case: A developer's app worked perfectly in testing until a user entered something unexpected and the AI returned nonsense. Because he'd added validation and tested edge cases before deploying, the app handled it gracefully instead of crashing in front of real users.

Action Drill: Test one part of your app with unusual and empty inputs and fix what breaks. Then deploy a simple app to a free hosting platform so it's live online.



04
Scaling, Capstone & Excellence

Module 7: Real-world projects | Module 8: Build your AI app

4.1 Real-world projects

Skill becomes credibility through real projects. Build things that solve genuine problems, for yourself, a community, or a small business, and put them on GitHub as your portfolio. Employers and clients care far more about what you've actually built than about certificates. A few solid, well-documented projects demonstrating engineering plus AI integration speak louder than any CV line.

Work on projects end-to-end: from idea and design through building, testing, deploying, and writing a clear README explaining what it does and how. Contributing to open-source or collaborating with others also builds real-world experience and network. Each project teaches new problems and grows your ability, and your body of work becomes your strongest career asset.

Field Example: A self-taught developer landed a job not through a degree but through three deployed projects on GitHub, including an AI-powered tool, each with a clear README. The interviewer said the working, documented projects made the decision easy. | Drill: Plan a portfolio project that solves a real problem using engineering + an LLM. Build it end-to-end and write a clear README for it on GitHub.

4.2 Build your AI app (Capstone)

Your capstone is a complete, working AI-powered application that you design, build, test and deploy yourself. Choose a real problem an LLM can genuinely help solve, keep the scope achievable, and take it all the way to a live, usable product. This ties together every skill in the book, engineering fundamentals, LLM understanding, prompting, APIs, app design, testing and deployment.

Expect challenges, integrating everything and handling real-world messiness is hard, and that struggle is exactly where mastery forms. Break it into steps, use documentation and AI assistants when stuck, and keep going until it runs for real users. When it's live, you'll have crossed from 'learning' to 'builder of AI software', with proof in hand and confidence earned.

Competency Area	Capstone Standard	Starting	Target Mastery
1. Clarity & Preparation	Real, deployed projects prove skill best.	Needs Draft	Confident & Ready
2. Execution Quality	Ship one complete, live AI-powered app.	In Practice	Portfolio-Grade
3. Real-World Proof	Design, build, test and deploy one complete AI-powered app this course. Take it all the way to live, then share the link and code as proof.	Planned	Completed & Reviewed
4. Consistency Habit	A deployed app is proof you're an AI builder.	Occasional	Weekly Discipline

5.1 Complete 8-Module Practice Matrix

Use this checklist to track every hands-on assignment in SWE & LLM Mastery from Module 1 to your final project:

Module	Focus Topic	Core Deliverable & Principle
Module 01	Software engineering basics	Engineering = reliable, maintainable, problem-solving code.
Module 02	Understanding LLMs	LLMs predict language, fluent but not always factual.
Module 03	Prompt engineering	Prompts are part of your app's code.
Module 04	Building with APIs	Your code calls LLMs and services via APIs.
Module 05	AI-powered app development	Use the LLM as one component of solid engineering.
Module 06	Testing and deployment	Test logic, user experience, and AI edge cases.
Module 07	Real-world projects	Real, deployed projects prove skill best.
Module 08	Build your AI app	Ship one complete, live AI-powered app.

5.2 Applied Fellowship Reflections

"Two coders build the same feature. One writes messy code that only they understand; the other writes clean, organised code with clear names and version control. Six months later, only the second can easily fix and extend it, that's engineering, not just coding."

CHRISCO Youth Aflame Cohort Case Note | Module 01





"A learner built and deployed a small app that summarises long documents into key points using an LLM API, with a clean interface and error handling. It was modest but real, live, and useful, and it became the project that opened doors for her."

CHRISCO Digital Academy Capstone Highlight | Module 08

06 90-Day Action Roadmap

True mastery in SWE & LLM Mastery is measured by what you build, ship, and steward after finishing the lessons. Follow this structured 90-day implementation plan developed by CHRISCO Youth Aflame (under CHRISCO Fellowship of Churches) to turn study into fruitfulness.

6.1 Four-Stage Post-Course Milestones

01

Days 1 to 14: Foundation Sprint

Complete Modules 1 to 4 drills and document your baseline specimen in SWE & LLM Mastery.

02

Days 15 to 30: First Live Output

Complete Modules 5 to 8 and ship your first real capstone deliverable: Design, build, test and deploy one complete AI-powered app this course. Take it all the way to live, then share the link and code as proof.

03

Days 31 to 60: Feedback & Iteration

Share your work with a mentor, peer, or real client/community audience and refine weak points.

04

Days 61 to 90: Service & Multiplication

Use your skill to serve your church, business, or clients faithfully and mentor one younger learner.

Follow-Up Stage	CHRISCO Support Channel	Contact & Cadence
Workbook Review	CHRISCO Digital Academy Desk	chriscoyouthaflame2025@gmail.com
Capstone Submission	CHRISCO Youth Aflame Mentors	shambetz@gmail.com
Peer Accountability	CHRISCO Fellowship Study Hubs	Monthly Cohort Check-In

CHRISCO Youth Aflame
CHRISCO DIGITAL ACADEMY | CHRISCO FELLOWSHIP OF CHURCHES

Equipping Youth, Elevating Purpose

CHRISCO Youth Aflame
CHRISCO Digital Academy
Under CHRISCO FELLOWSHIP OF CHURCHES

Official Gmail Contacts
chriscoyouthaflame2025@gmail.com
shambetz@gmail.com

Official Course E-Book for SWE & LLM Mastery (Software Engineering & AI Systems Track).
Prepared by CHRISCO Youth Aflame under CHRISCO FELLOWSHIP OF CHURCHES.
For course guidance or mentorship, email chriscoyouthaflame2025@gmail.com.